ApexPlanet Cybersecurity Internship - Task 5 Comprehensive Methodology

Author: Jetty Charan
Program: Cybersecurity & Ethical Hacking Internship
Timeline: Days 49 to 60
Target Environment: Metasploitable2 (`192.168.56.103`)
Attacker Platform: Kali Linux (`192.168.56.0/24` - VirtualBox Host-Only Network)

---

1. Governance, Scoping & Rules of Engagement

1.1 Methodology & Standards Alignment

The methodology followed throughout Task 5 aligns with industry-standard cyber testing and incident response frameworks:

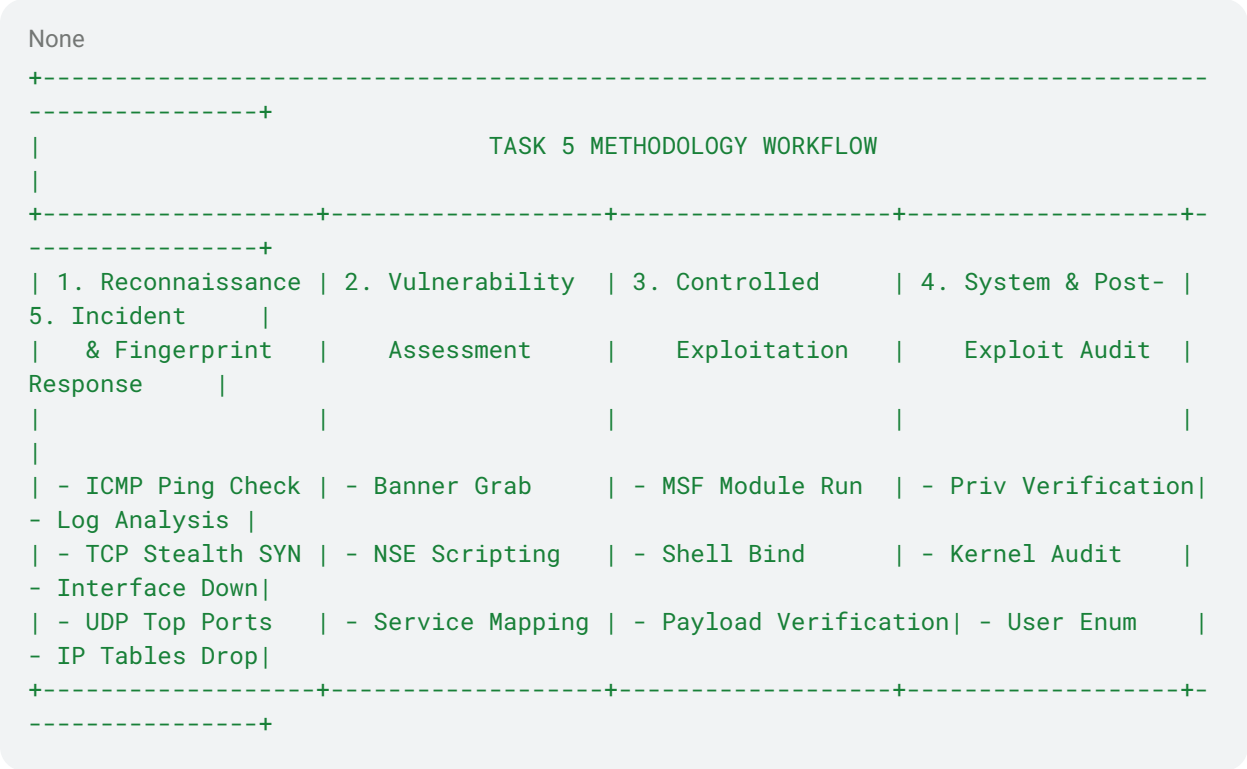
- **PTES (Penetration Testing Execution Standard):** Guided the pre-engagement, intelligence gathering, vulnerability analysis, exploitation, and post-exploitation reporting phases.
- **NIST SP 800-61 Rev. 2 (Computer Security Incident Handling Guide):** Guided the Incident Response (IR) phase, specifically covering Preparation, Detection & Analysis, Containment, Eradication, and Recovery.
- **OWASP & OSSTMM:** Provided principles for network service probing and cleartext security evaluation.

1.2 Boundary & Lab Rules of Engagement

- **In-Scope Target:** Single Linux VM node (`192.168.56.103`).
- **Attacker System:** Kali Linux running within an isolated VirtualBox Host-Only adapter.
- **Network Isolation:** No external internet routing or multi-tenant subnet interaction allowed during active testing to ensure total containment.

---

2. Phase-by-Phase Technical Methodology



Phase 1: Environment Readiness & Network Reachability

1. **Adapter Verification:** Ensure Kali Linux and Metasploitable2 reside on the same layer-2 VirtualBox Host-Only broadcast domain (`192.168.56.0/24`).
2. **Layer-3 Reachability Test:** Utilize ICMP Echo requests to confirm zero packet loss and establish baseline latency.

Phase 2: Reconnaissance & Service Fingerprinting

1. **TCP Stealth Scanning (`-sS`):** Send TCP SYN packets without completing the full 3-way handshake to map active listening ports quickly while minimizing log noise on legacy systems.
2. **Service Version Detection (`-sV`):** Interrogate open ports by matching service banners against known software fingerprints to determine exact software revisions (e.g., `vsftpd 2.3.4`, `OpenSSH 4.7p1`, `Apache 2.2.8`).
3. **OS Fingerprinting (`-O`):** Analyze TCP/IP stack implementation behavior to determine target operating system kernel details.
4. **UDP Service Probing (`-sU`):** Perform targeted sweeps against top UDP ports to identify stateless daemons (DNS, SNMP, NTP).

Phase 3: Vulnerability Analysis & Direct Banner Verification

1. **Manual Banner Grabbing:** Establish raw TCP streams using Netcat (`nc`) on ports 21 and 80 to verify if daemons leak specific build details in plain text prior to session authentication.
2. **Vulnerability Prioritization:** Map identified software releases against public vulnerability databases (CVE/NVD) to categorize findings by CVSS severity:
    - **Critical:** Remote Code Execution (RCE) via unauthenticated backdoors (e.g., `vsftpd 2.3.4`).
    - **High:** Insecure cleartext authentication protocols (Telnet, plain FTP).
    - **Medium:** Legacy protocol implementations with weak ciphers (SSH v1/legacy OpenSSH).

Phase 4: Controlled Exploitation & Validation

1. **Framework Initialization:** Launch Metasploit Framework (`msfconsole`) to manage exploit modularity and payload delivery safely.
2. **Exploit Target Configuration:** Match selected exploit module (`exploit/unix/ftp/vsftpd_234_backdoor`) with target parameters (`RHOSTS 192.168.56.103`).
3. **Execution & Session Verification:** Trigger payload to open a command shell session, confirming exploitability without causing target Denial of Service (DoS).

Phase 5: Post-Exploitation & System Auditing

1. **Privilege Level Determination:** Execute `id` and `whoami` within the active shell to determine whether execution context is privileged (`root`).
2. **Host Enumeration:** Collect kernel release details (`uname -a`), active hostnames (`hostname`), user databases (`/etc/passwd`), and local interface IP configurations (`ifconfig`).

Phase 6: Incident Response (IR) Simulation

1. **Detection & Log Analysis (Defensive Shift):**
    - Review authentication logs (`/var/log/auth.log`) and daemon logs (`/var/log/vsftpd.log`) to establish an intrusion timeline.
    - Query active network sockets (`netstat -tulnp`, `netstat -an`) to spot non-standard listening ports and established external connections.
2. **Containment:**
    - **Interface Isolation:** Bring down the main network interface (`ifconfig eth0 down`) to immediately sever active external connections and block lateral movement.
3. **Eradication & Hardening:**

- **Process Termination:** Identify spawned interactive shells (`ps aux | grep sh`) and terminate them forcefully (`kill -9`).
- **Firewall Enforcement:** Implement strict kernel firewall rules (`iptables -A INPUT -p tcp --destination-port 21 -j DROP`) to block inbound access on vulnerable ports during maintenance.

---

3. Summary Matrix of Methodological Steps

Phase	Core Objective	Primary Tool / Utility	Deliverable / Output
1. Readiness	Verify network communication	`ping`, `ifconfig`	Confirmed layer-3 link (`192.168.56.103`)
2. Recon	Map ports & daemon versions	`nmap`	Port scan logs & OS fingerprint
3. Probing	Manual banner inspection	`netcat`	Raw service response banners
4. Exploitation	Validate high-risk vulnerabilities	`msfconsole`	Active interactive session
5. Post-Audit	Verify impact & privileges	Interactive Shell (`id`, `uname`)	System privileges & host details
6. IR Detection	Identify compromise indicators	`netstat`, `/var/log/*`	Socket list & auth log anomalies
7. IR Containment	Isolate & block threat	`ifconfig`, `kill`, `iptables`	Terminated processes & drop rules